Identifying and Mitigating Flaky Tests in JavaScript

Negar Hashemi
Computational and Mathematical Sciences
Massey University
Palmerston North, New Zealand
n.hashemi@massey.ac.nz

2025 IEEE Conference on Software Testing, Verification and Validation (ICST) | 979-8-3315-0814-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICST62969.2025.10989021

***Abstract*—Flaky tests, which have non-deterministic outcomes (pass or fail when running on the same code), are a significant issue that affects software quality and makes it difficult to rely on test results. This research aims to explore the root causes of flaky tests in open-source JavaScript projects, develop practical techniques to manifest possible flaky behaviour, and provide a mechanism to mitigate some of those flaky tests.**

I. Problem Statement

Tests are considered flaky if they pass on some runs and fail on others when running on the same code version. Flaky tests are known to impact testing-related activities [1], [2], and also negatively affect product quality and delivery [3]–[5]. Indeed, flaky tests are problematic and present a significant challenge in the software industry [1], [2].

While flaky tests have been investigated in the scope of many languages and ecosystems, only a few studies have investigated flakiness in JavaScript, despite the language's popularity. JavaScript has been one of the most widely used programming languages in industry [6] and plays a key role in areas like web development, mobile apps, and the Internet of Things (IoT) [7], [8]. In addition, JavaScript uses asynchronous APIs extensively and allows for the development of platform-independent applications. Asynchronization (or other concurrency bugs) [9], [10] and platform dependency [10], [11], which are known sources of test flakiness in other languages, in particular, Java and Python. There is generally a lack of research on techniques and tools to deal with flaky tests in JavaScript.

This research examines the underlying causes of flakiness in real-world, open-source JavaScript projects. The study aims to identify patterns and factors that contribute to flaky tests and to analyze the strategies developers use to mitigate them. Once those causes are identified, the research will develop tools to detect and control flakiness in JavaScript projects for a selected number of the root causes.

The main objectives of this research are:

1) **Understanding flaky tests in JavaScript**: Investigate the root causes of flaky tests and the developers' response to flaky tests found in JavaScript projects.
2) **Detecting flaky test in JavaScript projects**: Based on the results of (1), we identify one of the causes of flakiness (i.e., test order dependency) and then develop a practical technique that aims to detect and manifest flaky tests.
3) **Controlling flaky tests in JavaScript projects**: Based on (1), we identify another cause of flaky tests (i.e., environmental dependency) and then design a technique to sanitize potential flaky tests from regression testing.

II. Research Questions

The first two objectives of this research have been accomplished as a result of an empirical study in open-source JavaScript programs [12] and an experiment on order-dependent tests [13]. Regarding the first objective, we attempted to answer the following research questions:

RQ1 What are the main causes of flaky tests in JavaScript projects?
RQ2 What strategies are followed when dealing with flaky tests in JavaScript projects?

The observations from the empirical study motivated the other two research objectives and questions. The results are explained in more detail in Section IV.

RQ3 How can order-dependent tests manifest their failures in JavaScript projects?
RQ4 Can we sanitize flaky tests caused by variations in environmental setups?

III. Research Approach

Regarding the first objective of the research, the top 40 most-starred JavaScript projects on GitHub were selected. They provide a large enough sample size (in terms of the number of commits) deemed suitable for our analysis. Using the GitHub API, we searched these projects for a set of keywords related to test flakiness. We selected a set of 452 commits and manually analysed the causes of flakiness as noted in them. This includes analysing the commit message, any code changes in the files in the repository, and the associated issues (if any) in the issue tracker. We also analysed the strategies followed when those flaky tests were fixed.

To answer RQ3 (Section II), we developed an approach to detect flaky tests caused by test order dependency, implemented in a new tool called JavaScript Test Order-dependency Detector (**`JS-TOD`**). The tool extracts all test paths from a given project and then reorders the tests and test suites using a user-defined number of reorders. `JS-TOD` will then run all tests (including the newly reordered tests) using a user-defined number of re-runs and finally export the results (i.e., test outcomes) into `JSON` files. We constructed a dataset of popular JavaScript projects with at least two tests (81 projects). We

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 06,2026 at 10:54:09 UTC from IEEE Xplore. Restrictions apply.

experimented with this approach using the selected projects, where we reordered each test suite using `JS-TOD` 10 times each and rerun them 10 times each. We then manually analysed each case that showed flaky outcomes to determine the cause of flakiness.

IV. Results and Observations

The results are organized based on the objectives of the research (Section I). We present the result of each of the three project's objectives below:

A. Understanding flaky tests in JavaScript

Our analysis showed that 70% of flaky test commits in JavaScript are caused by one of the following causes: *Concurrency*, *Async Wait*, *OS* or *Network*. Regarding how flaky tests are dealt with, most of the identified flaky tests (>80%) are fixed by developers to eliminate the flaky behaviour. However, other strategies developers follow include skipping the tests, quarantining the test for future fixes, or removing those tests.

An interesting observation from this study was that when comparing the proportion of flaky tests caused by test order dependency in JavaScript with those found in other languages, we observed a small number of order-dependent flaky tests in the studied JavaScript projects. Another observation from this empirical study is that *platform* (e.g., running a test in a specific or using specific environmental variables) is found to be one of the top three causes of flakiness in JavaScript, unlike previous studies on test flakiness in other languages ([9], [11]). This includes all scenarios where the test runs in different environments, browsers, or OS.

B. Detecting flaky test in JavaScript projects

As shown in Section IV-A, we found that the number of flaky tests caused by order dependency is low compared to other languages such as Java [9] and Python [14]. Test order dependency occurs when the tests' outcomes depend on the order in which they are executed. Flaky tests that pass in one order but fail in another are known as *order-dependent tests*. However, the findings from our empirical study were based on the reported flaky tests in open-source projects (i.e., extracted from issue trackers). The projects may contain order-dependent tests that the developers have not yet discovered, potentially failing and misleading developers later during development.

We investigated order-dependent tests in JavaScript projects using Jest as the testing framework. Jest is the most popular JavaScript testing framework [15], [16]. Jest offers many features, such as parallelization and ordered test execution, but tests may still share states, leading to order-dependent tests.

This study aimed to understand the causes and possible detection mechanisms of order-dependent tests in JavaScript projects. The study also investigates the reasons for the test failures, i.e., what state these tests share between them. The results can provide better insights into how developers manage order-dependent tests in their test suites, debugging their root causes to fix them eventually. Hence, they eliminate variation in test outcomes when the test order changes.

Our evaluation of 81 projects showed a notably low number of order-dependent tests in JavaScript projects compared to other studied languages, particularly Python [14], [17] and Java [9], [18]. We found 55 order-dependent test instances in 20 test suites across 10 projects.

In addition, the result showed that the main causes of the order-dependent flaky tests are shared state between tests. This is represented in two forms: 1) shared files (e.g., access to a shared file created in one of the tests and accessed by other tests) and 2) shared mocking state (e.g., a mocking state changes between tests). A shared mocking state is a new cause of order-dependent tests that has not been reported previously in the context of other languages but was one of the common causes of order-dependent flakiness in JavaScript.

V. Research Plan and Current Work

Similar to the observations from our empirical study, Costa et al. [19] identified platform dependency as a major cause of flaky tests in JavaScript projects. Consequently, having automated methods to detect and manage such flaky tests would benefit developers. Some testing frameworks have provided mechanisms to run environment-specific tests (e.g., JUnit annotation for OS conditional tests [20]). We aim to develop a technique to mitigate and control environmental flakiness in JavaScript projects.

We will construct a dataset of open-source JavaScript projects from GitHub to better understand environmental test flakiness and how developers address it. The selection criteria include the number of GitHub stars and the presence of at least one test. For each project in the dataset, we will analyze the results under two conditions: (1) running the tests in different environments (including OS, browser, and test environment) to observe if they fail, and (2) examining the test code statically to determine whether environment-specific options are used.

These observations will build on the findings from the empirical study (Section IV-A), which showed developers use different methods to deal with flakiness in JavaScript. Sometimes, developers add suitable assumptions to the tests to run the tests on specific environments or configurations.

We aim to develop a tool to sanitize the environmental flakiness by intercepting the execution of tests, checking their state, and then inferring assumptions. We will utilize a similar approach that has been developed to sanitize tests with network dependencies [21]. The proposed approach will add assumptions to the tests to check the execution environment that cannot be checked using a unit testing API. Those assumptions sanitise the test and *skip* the test rather than failing it. When the state of a test is changed to *skip*, an appropriate message, including why the test was skipped, will be added to the generated reports to inform the users.

Acknowledgments

I would like to thank my supervisors and collaborators, Amjed Tahir, Rachel Blagojevic, Shawn Rasheed, and August Shi, for contributing to this work.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 06,2026 at 10:54:09 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] M. Machalica, A. Samylkin, M. Porth, and S. Chandra, "Predictive test selection," in *Proceedings of the International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019.

[2] W. Lam, A. Shi, R. Oei, S. Zhang, M. D. Ernst, and T. Xie, "Dependent-test-aware regression testing techniques," in *Proceedings of the ACM International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020.

[3] M. Fowler, "Eradicating non-determinism in tests," 2011, https://martinfowler.com/articles/nonDeterminism.html.

[4] A. Sandhu, "How to fix flaky tests," 2015. [Online]. Available: https://tech.justeattakeaway.com/2015/03/30/how-to-fix-flaky-tests/

[5] J. Palmer, "Test flakiness – methods for identifying and dealing with flaky tests," 2019. [Online]. Available: https://engineering.atspotify.com/2019/11/18/test-flakiness-methods-for-identifying-and-dealing-with-flaky-tests/

[6] "Stackoverflow developer survay 2024," https://survey.stackoverflow.co/2024/, (Accessed on 09/02/2024).

[7] S. Technologies, "Why is javascript so popular?" 2018. [Online]. Available: https://web.archive.org/web/20191208013733/https://www.simplytechnologies.net/blog/2018/4/11/why-is-javascript-so-popular

[8] I. Lima, J. Silva, B. Miranda, G. Pinto, and M. d'Amorim, "Exposing bugs in javascript engines through test transplantation and differential testing," *Software Quality Journal*, vol. 29, no. 1, pp. 129–158, 2021.

[9] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the ACM International Symposium on Foundations of Software Engineering (FSE)*. ACM, 2014.

[10] S. Thorve, C. Sreshtha, and N. Meng, "An empirical study of flaky tests in android apps," in *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2018.

[11] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2019.

[12] N. Hashemi, A. Tahir, and S. Rasheed, "An empirical study of flaky tests in javascript," in *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2022.

[13] N. Hashemi, A. Tahir, S. Rasheed, A. Shi, and R. Blagojevic, "Detecting and evaluating order-dependent flaky tests in javascript," in *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST) - (Accepted, to appear)*. IEEE, 2025.

[14] M. Gruber, S. Lukasczyk, F. Kroiß, and G. Fraser, "An empirical study of flaky tests in python," in *Proceedings of the IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2021.

[15] M. Taleb, "Javascript unit testing frameworks in 2024: A comparison · raygun blog," https://raygun.com/blog/javascript-unit-testing-frameworks/, 2023, (Accessed on 09/02/2024).

[16] "State of javascript 2023: Testing," https://2023.stateofjs.com/en-US/libraries/testing/, (Accessed on 09/02/2024).

[17] R. Wang, Y. Chen, and W. Lam, "Ipflakies: A framework for detecting and fixing python order-dependent flaky tests," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. ACM, 2022.

[18] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "idflakies: A framework for detecting and partially classifying flaky tests," in *Proceedings of the IEEE Conference on Ssoftware Testing, Validation and Verification (ICST)*, IEEE. IEEE, 2019.

[19] K. Costa, R. Ferreira, G. Pinto, M. d'Amorim, and B. Miranda, "Test flakiness across programming languages," *IEEE Transactions on Software Engineering*, 2022.

[20] "org.junit.jupiter.api.condition (junit 5.2.0 api)," [Online; accessed 2024-12-20]. [Online]. Available: https://junit.org/junit5/docs/5.2.0/api/org/junit/jupiter/api/condition/package-summary.html

[21] J. Dietrich, S. Rasheed, and A. Tahir, "Flaky test sanitisation via on-the-fly assumption inference for tests with network dependencies," in *Proceedings of the IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2022.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 06,2026 at 10:54:09 UTC from IEEE Xplore. Restrictions apply.